

智溯 · 多智能体科研协同编辑与版权溯源系统

软件使用说明书

面向科研诚信的多 Agent 实时协同与版权溯源系统 (Multi-Agent Collaboration & AIGC Provenance System)
软件版本：V1.0.0

一、软件概述

1.1 开发背景与目的

随着人工智能生成内容 (AIGC) 在科研写作中的广泛应用，学术不端风险、AI 代写识别难、协同编辑过程不可追溯等问题日益突出。传统在线文档工具虽然支持多人协同，但普遍存在以下缺陷：

问题	传统工具的不足
协同过程不可信	无法区分 AI 生成内容与人类创作内容，无法定位内容来源
版权归属难追溯	缺少对编辑历史的不可篡改记录，发生抄袭纠纷时缺乏证据链
科研辅助割裂	文献检索、论文写作、学术规范审查相互分离，需要切换多个工具
水印检测缺失	无法可靠判断一段文本是否由 AI 大模型生成

本软件面向科研写作场景，将实时协同编辑、多智能体科研辅助、AIGC 水印检测与版权溯源四大能力整合于一体，为科研团队提供可信、可溯源、可审计的协同写作环境，服务于科研诚信建设与 AIGC 可信溯源。

1.2 软件名称与版本

- 软件全称：智溯 · 多智能体科研协同编辑与版权溯源系统
- 软件简称：智溯
- 版本号：V1.0.0
- 英文名称：Multi-Agent Collaboration & AIGC Provenance System

1.3 主要功能

编号	功能模块	说明
1	实时协同编辑	基于 Yjs CRDT 的多端实时编辑，多光标互见，AI/人类作者内容着色区分
2	多智能体科研辅助	Research (文献检索)、Writer (论文起草)、Supervisor (学术规范审查) 三类 Agent 协同工作
3	AIGC 水印检测	基于 Kirchenbauer 绿名单算法的 AI 生成内容检测，检测动作留痕可追溯
4	版权溯源链	操作日志 SHA-256 哈希链，任一历史操作被篡改即整链失效
5	文献检索与引用	关键词检索内置文献库，一键生成 GB/T 7714 引文并插入文档
6	文档导出	导出含溯源元数据的 Markdown 文件 (标题/正文/溯源校验状态/参考文献)

1.4 适用范围

适用于高校科研团队、课题组、期刊编辑部的论文协同撰写与审稿场景；也可用于需要对 AI 生成内容进行检测、对编辑历史进行审计的一般写作场景。

二、运行环境

2.1 开发环境

类别	要求
操作系统	Windows 10/11、macOS、Linux
处理器	Intel/AMD 64 位，双核及以上
内存	建议 8 GB 及以上
硬盘	建议 100 GB 及以上可用空间
后端语言	Python 3.10+
前端语言	Node.js 18+（npm）
开发工具	Visual Studio Code、Git
数据库	PostgreSQL 14+（可选，未安装时自动回退 SQLite）

2.2 运行环境

类别	要求
操作系统	Windows / macOS / Linux
客户端	现代浏览器（Chrome / Edge 最新版，需支持 WebSocket）
后端运行时	Python 3.10+、FastAPI
内存	建议 4 GB 及以上
网络	支持 HTTP 与 WebSocket 连接（局域网/互联网均可）

2.3 兼容性说明

- 数据库自动回退机制：优先连接 PostgreSQL，连接失败时自动使用 SQLite，无需任何数据库安装即可运行，保证最低环境下的可用性；
- 文献检索自动降级机制：arXiv 实时检索不可用（无网络/超时）时自动回退到内置本地文献库，演示链路始终可用。

三、安装与部署

3.1 后端服务安装

在项目 backend 目录执行：

```
cd backend
```

```
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # macOS/Linux
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8000
```

首次启动自动完成：创建数据库表、写入演示用户与演示文档、写入 8 条文献种子语料。

3.2 前端安装

在项目 frontend 目录执行：

```
cd frontend
npm install
npm run dev
```

浏览器访问 <http://localhost:5173> 即进入系统主界面。Vite 已配置代理，将 /api 与 /ws 请求转发至 localhost:8000 后端服务。

3.3 数据库配置

- 默认使用 SQLite：数据库文件为 backend/research_colab.db，零配置运行；
- 若环境变量 DATABASE_URL 指向可连接的 PostgreSQL 实例，则自动切换为 PostgreSQL，建表脚本见 database/init.sql（可选）。

3.4 启动与健康检查

启动后依次执行以下检查确认部署成功：

```
curl http://localhost:8000/api/health    # 返回 {"status":"ok",...}
curl http://localhost:8000/api/bootstrap # 返回演示用户与文档列表
```

3.5 运行测试

```
cd backend
python -m pytest tests/ -q
```

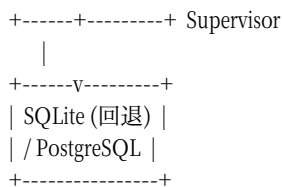
全部用例通过即视为部署正确（当前共 29 个用例，覆盖哈希链校验、水印检测、学术规范审查、API 集成等）。

四、系统架构与技术特点

4.1 总体架构

系统采用前后端分离的浏览器/服务器架构：





- 前端：React 18 + TypeScript + Vite 5 + TailwindCSS；Tiptap 编辑器通过 @tiptap/extension-collaboration 接入 Yjs CRDT，y-websocket 负责实时同步，y-protocols/awareness 实现多光标；
- 后端：FastAPI + SQLAlchemy (async)；WebSocket 端点为纯转发模式——服务端不持有 CRDT 状态，只维护房间连接列表并按 y-websocket 协议转发增量更新与 awareness，客户端之间直接收敛；
- 数据库：优先 PostgreSQL (asyncpg)，未安装时自动回退 SQLite (aiosqlite)。

4.2 协同编辑原理 (Yjs CRDT)

协同编辑采用 Yjs CRDT（无冲突复制数据类型）实现：

1. 每个客户端持有完整文档状态，编辑产生增量更新 (update)；
2. 更新经 WebSocket 发送至服务端 /ws/{doc_id} 端点，服务端按 y-websocket 协议原样转发给同一房间的其他客户端（排除发送者）；
3. 客户端通过 y-protocols/awareness 广播光标位置，实现多光标互见；
4. 连接复用：前端以模块级单例缓存保证同一文档全页面共享同一 Y.Doc 与同一条 WebSocket 连接，避免重复连接与 awareness 无限回环。

协议关键细节（项目踩坑修复后的正确实现）：

- update / state vector 以 varUint8Array 编码 (LEB128 变长长度前缀)；
- 空 update 必须编码为 [0x00, 0x00]，否则客户端 applyUpdate 会抛异常；
- SYNC_STEP1 阶段服务端回复空 update，客户端进入 synced 状态；
- AWARENESS 原样转发，排除发送者，不回显。

4.3 AIGC 水印算法 (Kirchenbauer 绿名单水印)

水印引擎为 Kirchenbauer 绿名单水印 (KGW) 的论文级实现 (Kirchenbauer et al., "A Watermark for Large Language Models", ICML 2023, arXiv:2301.10226)：

- 绿名单生成：以哈希密钥与前序 token 的乘积为随机种子，对词表洗牌 (randperm) 取前 γ 比例为绿名单；检测端以相同种子还原，密钥经 SHA-256 混合，不可被他人预测；
- 统计检验：计算 z-score 与 p-value，判定规则 $z > 4.0$ (论文建议)，置信度取 $1 - p\text{-value}$ ；
- 安全检测：默认对重复 bigram 去重计数，防止文本重复片段虚高统计量；
- 接入方式：默认字符级 tokenizer 演示闭环，预留 tiktoken / transformers 真实 tokenizer 接入点。

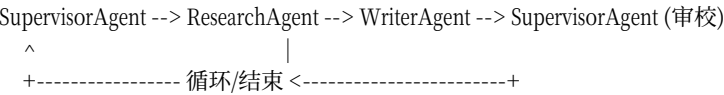
4.4 版权溯源哈希链

每次操作生成 OpLog，通过 SHA-256 链接：

```
current_hash = sha256(prev_hash + operation_json + timestamp)
```

任一历史记录被篡改，整条链验证失败。系统提供链完整性校验接口，返回 {"valid": true} 或标记篡改位置。

4.5 多 Agent 编排



- ResearchAgent：从自然语言指令自动抽取英文关键词，优先经 arXiv API 实时检索真实论文，无网络/失败时降级到本地文献库，结果标注来源；
- WriterAgent：基于检索结果生成论文草稿段落，写入编辑器正文（自动标记为 AI 作者）；
- SupervisorAgent：调用学术规范静态审查引擎，对草稿执行篇幅、GB/T 7714 引用编号连续性、参考文献章节与条数匹配等检查，输出结构化导师审稿意见。

五、主界面说明

5.1 主界面布局

主界面为三栏布局：

区域	内容
顶栏	系统标识、文档切换下拉框、"新建文档"按钮、"导出 Markdown"按钮、当前用户标识
左侧栏（Agent 面板）	文献检索 Agent、论文写作 Agent、导师审稿 Agent 的触发按钮与消息流
中间栏	协同编辑器主区域（Tiptap 富文本编辑器）
右侧栏（功能面板）	三个标签页——"水印检测"、"溯源链"、"文献检索"

5.2 编辑器作者着色

- AI Agent 生成的内容：蓝色背景显示；
- 人类作者输入的内容：白色背景显示；
- 通过 ProseMirror Decoration 插件将 Yjs 文本 delta 属性映射为视图层背景色。

六、功能操作指南

6.1 文档管理

1. 在顶栏下拉框选择已有文档，或点击"新建文档"创建新文档；2. 文档内容自动保存至服务端数据库；
3. 切换文档时系统自动关闭旧文档的协同连接，避免资源泄漏。

6.2 实时协同编辑

1. 在编辑器内直接输入内容，编辑器自动保存至服务端；2. 打开两个浏览器标签页访问同一文档，任一标签页中的输入会实时出现在另一标签页，双方光标（多光标）互相可见；3. 文档中 AI Agent 生成的内容以蓝色背景显示，人类作者输入的内容以白色背景显示，便于区分内容来源；4. 支持富文本编辑能力（标题、加粗、斜体、列表等，由 Tiptap Starter Kit 提供）。

6.3 多智能体科研辅助（Agent）

在左侧 Agent 面板中：

1. 启动文献检索：在输入框填写研究主题（如"AI 水印与版权溯源"），点击按钮。系统自动抽取英文关键词，优先通过 arXiv API 实时检索真实论文（标题、作者、年份、分类目录齐全），无网络或检索失败时自动回退到本地文献库，并在结果中标注来源: arXiv 实时检索 或 来源: 本地文献库；2. 启动论文写作：基于检索结果生成论文草稿段落，并写入编辑器正文（该段内容自动标记为 AI 作者，蓝色背景显示）；3. 导师审稿：对当前草稿执行学术规范静态检查，输出审稿意见：
 - 通过：草稿篇幅、引用编号、参考文献章节等符合要求；
 - 需修改：列出具体问题（如篇幅过短、引用编号不连续、缺少参考文献章节等）。

6.4 AIGC 水印检测

1. 切换右侧面板至"水印检测"；2. 点击"检测当前文档全文并留痕"：检测整篇文档是否包含 AI 水印，并自动生成一条检测历史记录（该动作同时写入溯源哈希链，可被审计）；3. 也可在文本框中粘贴任意文本，点击"检测粘贴的文本"进行独立检测；4. 检测结果以"人类创作 / AI 生成（含水印）"徽标与置信度进度条展示，下方列表展示历史检测记录（检测时间、模型、参数）。

6.5 版权溯源链

1. 切换右侧面板至"溯源链"；2. 面板展示该文档全部操作日志（插入 / 替换 / AI 生成 / 水印检测），每条记录含哈希值摘要，按时间正序排列并连线；3. 顶部徽标显示整条链的校验结果："✓ 链条完整 · 未被篡改"或"检测到篡改！"，并显示已校验的记录条数；4. 点击"刷新"按钮重新加载并重新校验。

6.6 文献检索与引用插入

1. 切换右侧面板至"文献检索"；2. 输入关键词（如"CRDT"）并搜索，结果列表展示文献标题、作者、年份、来源；3. 点击某条文献的"插入引用"按钮，系统生成 GB/T 7714 格式引文并插入编辑器正文光标处；4. 插入的引文自动标记为人类作者内容（白色背景），并进入溯源链记录。

6.7 文档导出

1. 点击顶栏"导出 Markdown"按钮；2. 浏览器下载文档标题.md 文件，内容包含：文档标题、导出时间、字数统计、溯源哈希链校验状态、正文（自动去除编辑格式标签）、水印检测记录、GB/T 7714 参考文献列表；3.

该导出文件可直接作为论文底稿或版权证明材料使用。

七、常见问题与处理

问题	处理方式
页面提示"后端服务未启动"	确认后端已在 8000 端口运行，curl http://localhost:8000/api/health 返回 ok
编辑器内容刷新后丢失	内容已保存在服务端数据库，刷新页面后从数据库恢复；多端实时同步依赖 WebSocket 连接
水印检测返回"人类创作"	当前文档由人工输入时属正常；AI 生成内容（蓝色段落）才带水印信号
溯源链校验显示失败	表示历史日志被修改，系统已成功识别篡改行为（预期行为）
文献检索无结果	可能处于无网络环境，系统已自动回退本地文献库；请检查网络连通性
端口占用	修改 uvicorn 端口与 frontend/vite.config.ts 中的代理目标保持一致
协同编辑不同步	检查浏览器是否禁用 WebSocket；刷新页面重连 /ws/{doc_id}

八、系统数据结构

系统采用关系型数据库存储业务数据，核心表结构如下（PostgreSQL 方言为原生 UUID/JSONB/BYTEA，SQLite 自动降级为 CHAR(32)/JSON/BLOB）：

8.1 用户表（users）

字段	类型	说明
id	UUID	主键
username	VARCHAR(64)	用户名（唯一）
email	VARCHAR(255)	邮箱（唯一）
display_name	VARCHAR(128)	显示名称
avatar_url	TEXT	头像地址（可空）
role	VARCHAR(32)	角色：admin / researcher / collaborator / ai_agent
created_at / updated_at	TIMESTAMP	创建/更新时间

8.2 文档表（documents）

字段	类型	说明
id	UUID	主键
title	VARCHAR(512)	文档标题
owner_id	UUID	所有者（外键 → users.id）
yjs_state	BYTEA	Yjs CRDT 状态快照（二进制，可空）
content	TEXT	明文快照，供预览与全文检索

字段	类型	说明
watermark_status	SMALLINT	0-未检测 1-含AI水印 2-纯人类创作
agent_session_id	UUID	关联的 Agent 会话（可空）
created_at / updated_at	TIMESTAMP	创建/更新时间

8.3 文档-协作者关联表（document_collaborators）

字段	类型	说明
id	INTEGER	主键（自增）
document_id	UUID	文档（外键）
user_id	UUID	用户（外键）

（document_id + user_id 唯一约束，实现文档与协作者的多对多关系）

8.4 操作日志表（op_logs，版权溯源哈希链）

字段	类型	说明
id	UUID	主键
doc_id	UUID	文档（外键）
user_id	UUID	操作者（外键）
op_type	VARCHAR(32)	insert / delete / replace / ai_generate / watermark_checked
operation	JSONB	操作内容 JSON
prev_hash	VARCHAR(64)	前一条日志的 current_hash（链首为空串）
current_hash	VARCHAR(64)	当前哈希（唯一）：sha256(prev_hash + operation + created_at)
created_at	TIMESTAMP	操作时间

8.5 水印记录表（watermark_records）

字段	类型	说明
id	UUID	主键
doc_id	UUID	文档（外键）
model_name	VARCHAR(128)	水印模型名称
gamma	FLOAT	绿名单比例（默认 0.5）
delta	FLOAT	logits 偏移强度（默认 2.0）
secret_key	BYTEA	哈希密钥（用于还原绿名单）
token_seq	JSONB	涉水印 token 序列（调试用）
created_at	TIMESTAMP	记录时间

8.6 文献资源表（literature）

字段	类型	说明
id	UUID	主键
title	VARCHAR(512)	文献标题
authors	VARCHAR(512)	作者
year	INTEGER	发表年份
source	VARCHAR(128)	来源
abstract	TEXT	摘要
keywords	VARCHAR(256)	关键词
url	TEXT	链接（可空）
created_at	TIMESTAMP	入库时间

8.7 权限配置表（permission_configs）

字段	类型	说明
id	UUID	主键
doc_id	UUID	文档（外键，唯一）
collab_mode	VARCHAR(16)	open（公开协作） / invited（受邀协作）
watermark_policy	VARCHAR(16)	enforce（强制水印） / optional（可选）
export_policy	VARCHAR(16)	allow / deny
updated_at	TIMESTAMP	更新时间

九、系统接口清单

系统对外提供 RESTful 接口与 WebSocket 实时通道，主要接口如下：

接口路径	方法	说明
/api/health	GET	服务健康检查
/api/bootstrap	GET	初始化演示用户与演示文档
/api/documents	GET	文档列表
/api/documents	POST	新建文档
/api/documents/{doc_id}	GET	获取文档详情
/api/documents/{doc_id}	DELETE	删除文档
/api/documents/{doc_id}/export	GET	导出 Markdown（含溯源元数据）
/api/documents/{doc_id}/yjs-state	GET / PUT	Yjs 协同状态读取 / 保存
/api/agents/invoke	POST	触发 Agent（文献检索 / 论文写作 / 导师审稿）
/api/agents/sessions/{sid}/state	GET	获取 Agent 会话状态

接口路径	方法	说明
/api/agents/sessions/{sid}/messages	GET	获取 Agent 消息流
/api/watermark/detect	POST	检测任意文本的水印信号
/api/watermark/documents/{doc_id}/detect	POST	检测文档全文并留痕（写入哈希链）
/api/watermark/documents/{doc_id}/records	GET	水印检测历史记录
/api/watermark/documents/{doc_id}/provenance	GET	版权溯源链操作日志
/api/watermark/documents/{doc_id}/provenance/verify	GET	溯源链完整性校验
/api/literature/search	GET	文献关键词检索
/api/literature/{lit_id}	GET	文献详情
/api/literature/{lit_id}/citation	GET	生成 GB/T 7714 格式引文
/ws/{doc_id}	WebSocket	协同编辑实时同步通道（y-websocket 协议，纯转发模式）

十、开源代码与许可

本项目科研核心模块参考/适配了以下开源成果，版权声明保留于
THIRD_PARTY_NOTICES.md，各许可证全文存放于 THIRD_PARTY/ 目录：

开源项目	用途	许可证
[jwkirchenbauer/lm-watermarking](https://github.com/jwkirchenbauer/lm-watermarking)	Kirchenbauer 水印算法（绿名单生成 / z-score / p-value / bigram 去重检测）	Apache-2.0
[THU-BPM/MarkLLM](https://github.com/THU-BPM/MarkLLM)	水印算法扩展参考（KGW 家族实现与鲁棒性评估方法）	Apache-2.0
[arXiv API](https://export.arxiv.org/api/query)	文献检索 Agent 的实时论文数据源	arXiv 公开接口

十一、版本信息

- 软件版本：V1.0.0
- 技术栈：React 18 + TypeScript + Vite + Yjs（前端）；FastAPI + SQLAlchemy 2.0（后端）
- 核心算法：Yjs CRDT 实时协同、Kirchenbauer 绿名单水印（KGW，论文级实现）、SHA-256 操作日志哈希链
- 编程语言：Python、TypeScript / JavaScript
- 源程序量：约 6500 行
- 开发者：长安大学 大创项目组